CS1312
Algorithms Design and Analysis

Sorting

(Insertion sort & Merge sort)

Sorting Problem

- Arranging elements of set into order (ascending , descending).

- Algorithm design techniques:
  - Incremental
  - Divide and Conquer

- Examples:
  - Insertion Sort
  - Merge Sort
  - Heap Sort
  - Counting Sort
  - Radix Sort
  - Bucket Sort

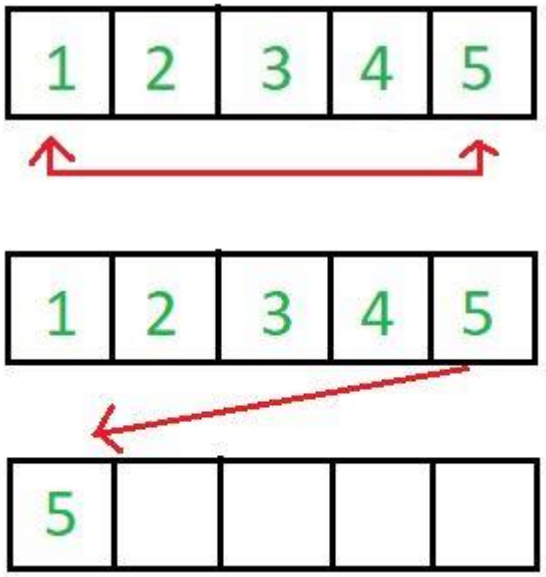
Sorting applications

- For searching on unsorted data by comparing keys, optimal solutions require $\theta(n)$ comparisons.
- For searching on sorted data by comparing keys, optimal solutions require $\theta(\log n)$ comparisons. E.g. BST

In place

- **In-place algorithms** rearrange the elements **within the array**, with at most a constant number stored outside the array at any time.

- Examples:
  - Selection sort
  - Bubble sort
  - Insertion sort
  - Heapsort



Stability

- **Stable algorithms** are the algorithms in which their elements appear in the output array in the same order as they do in the input array.
- If the same element is present multiple times, then they retain the original relative order of positions.



Unsorted array

- Examples:
  - Bubble sort.
  - Insertion sort.
  - Merge sort.

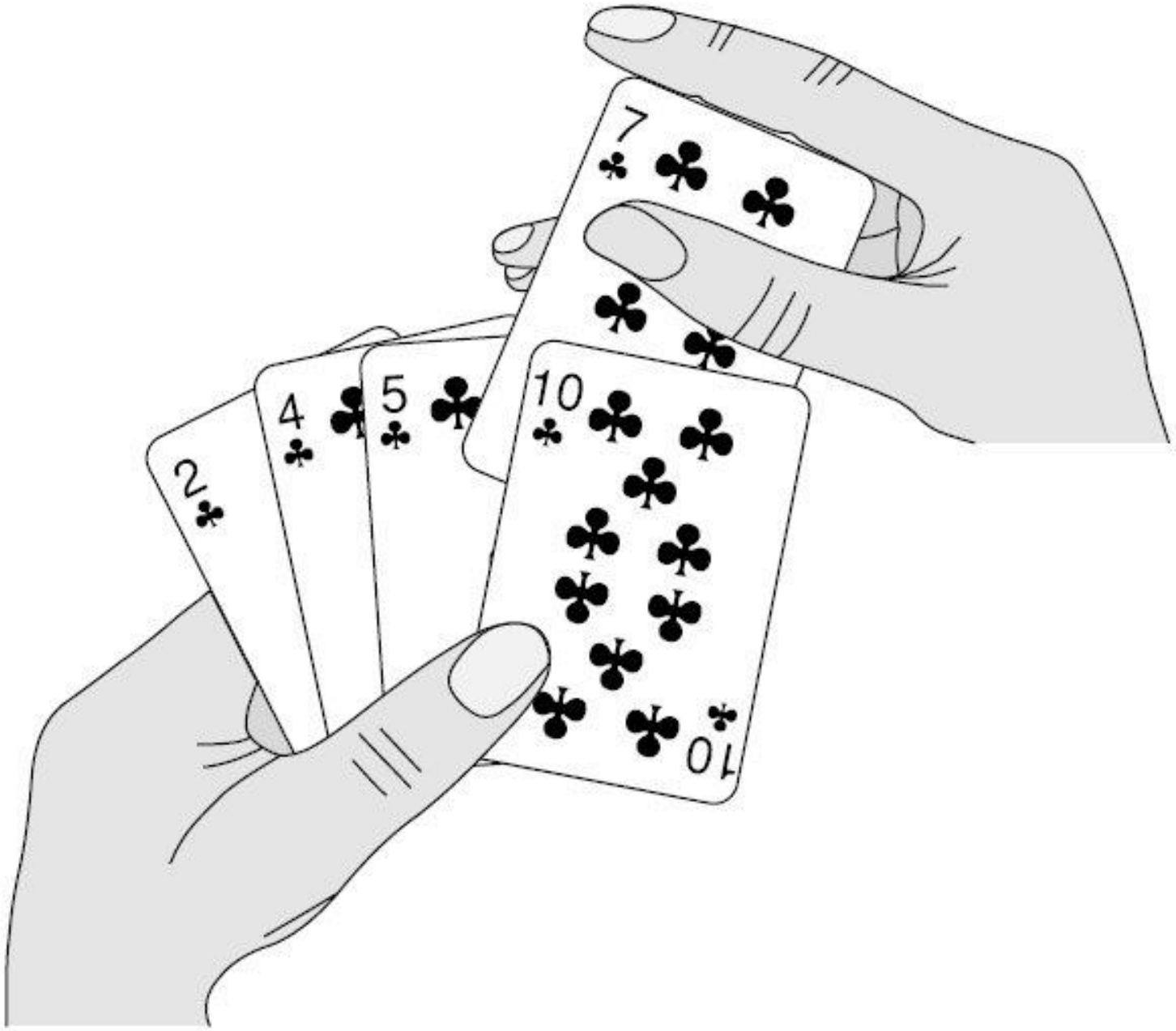


Stable sort



Unstable sort

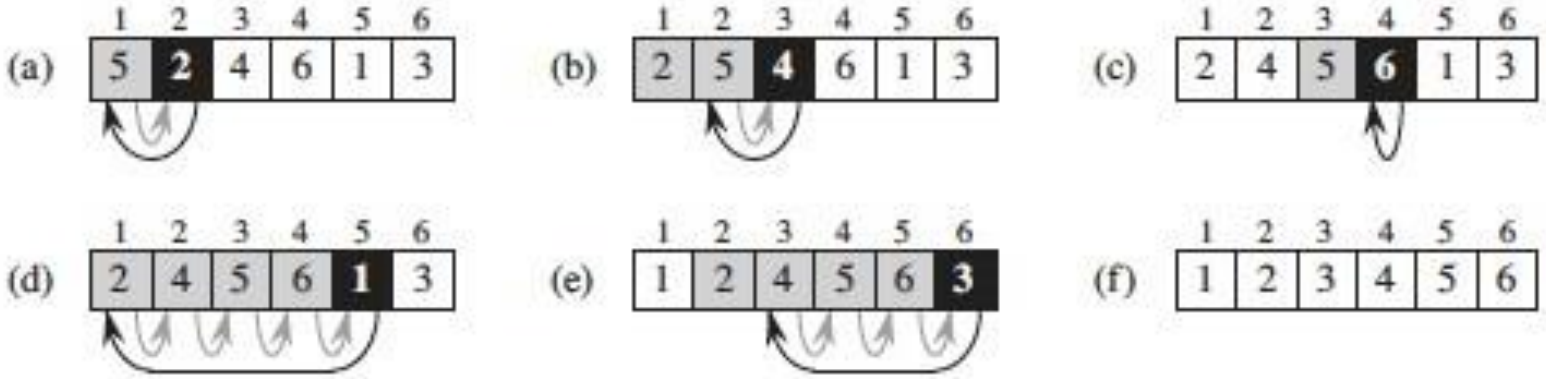
Insertion Sort


7
2
4
5
10
10

Insertion Sort

- Insertion of an element in proper order:
  - Begin with a sequence E of n elements in arbitrary order.
  - Initially assume the sorted segment contains first element.
  - Let x be the next element to be inserted in sorted segment, pull x “out of the way”, leaving a vacancy.
  - Repeatedly compare x to the element just to the left of the vacancy, and as long as x is smaller, move that element into the vacancy,
  - else put x in the vacancy,
  - Repeat for the next element that has not yet examined.

Insertion Sort



Insertion Sort

INSERTION-SORT(A)	*cost*	*times*
1 **for** $j = 2$ **to** $A.length$	c_1	n
2 $key = A[j]$	c_2	$n-1$
3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4 $i = j-1$	c_4	$n-1$
5 **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n} (t_j - 1)$
7 $i = i-1$	c_7	$\sum_{j=2}^{n} (t_j - 1)$
8 $A[i+1] = key$	c_8	$n-1$

Insertion Sort

- Insertion sort running time:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2} t_j + c_6 \sum_{j=2} (t_j - 1)$$

$$+ c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1) .$$

- The **best case** occurs if the array is already sorted:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

- It could be expressed as :

  $an + b$ (linear function of n)

Insertion Sort

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1 \qquad \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$$

- The **worst case** occurs if the array is in reverse order:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8) \, . \end{aligned}$$

Insertion Sort

- The **average case** is often roughly as bad as the worst case.
- On average, half the elements in A [1 … j-1] are less than A[j], and half the elements are greater.
- $t_j = ½\ j$
- The resulting average-case running time turns out to be a quadratic function of the input size.

Merge Sort

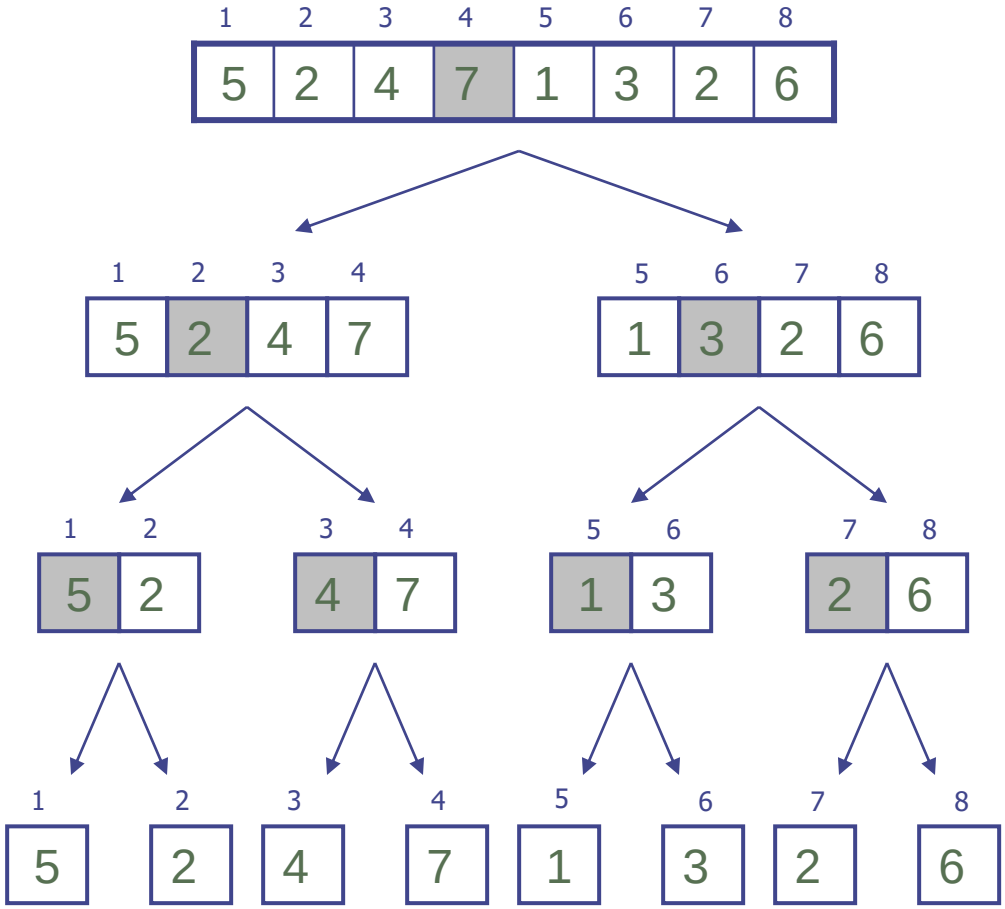
Merge Sort

To sort an array A[p . . r]:

- **Divide**
  - Divide the n-element sequence to be sorted into two subsequences of n/2 elements each
- **Conquer**
  - Sort the subsequences recursively using merge sort.
  - When the size of the sequences is 1 there is nothing more to do.
- **Combine**
  - Merge the two sorted subsequences.

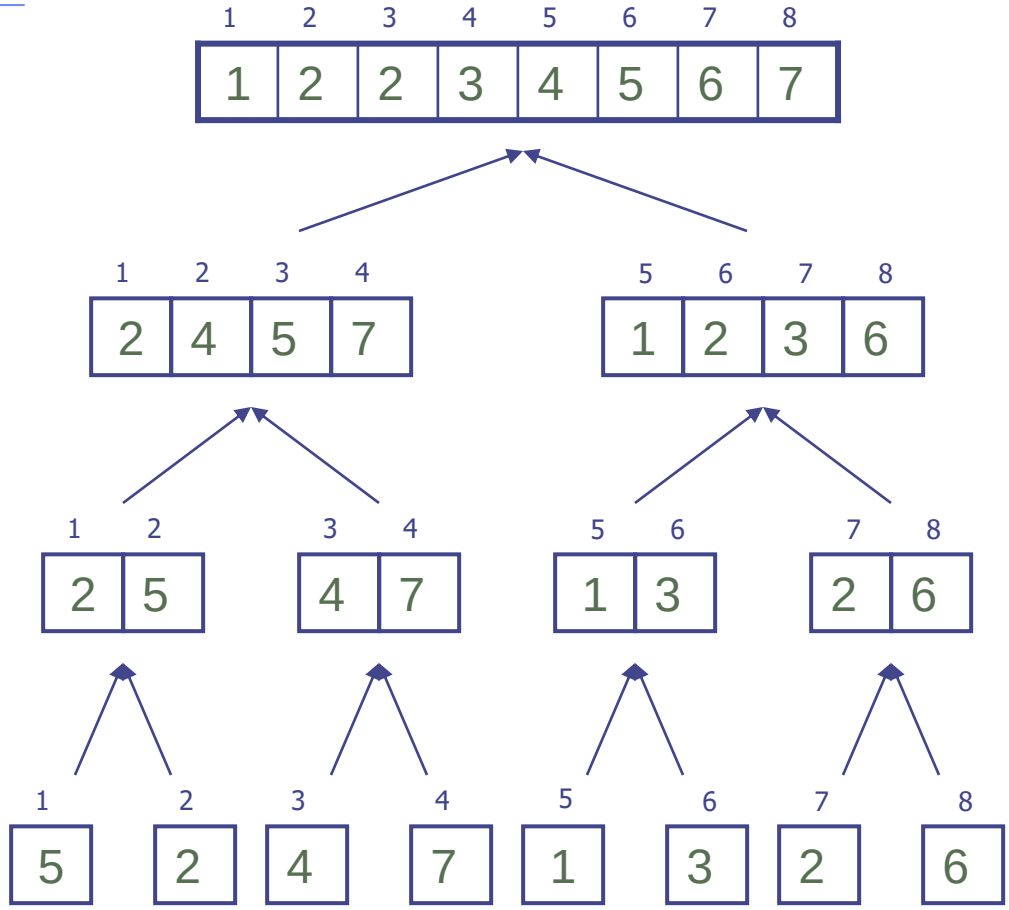
Example – n Power of 2

Divide



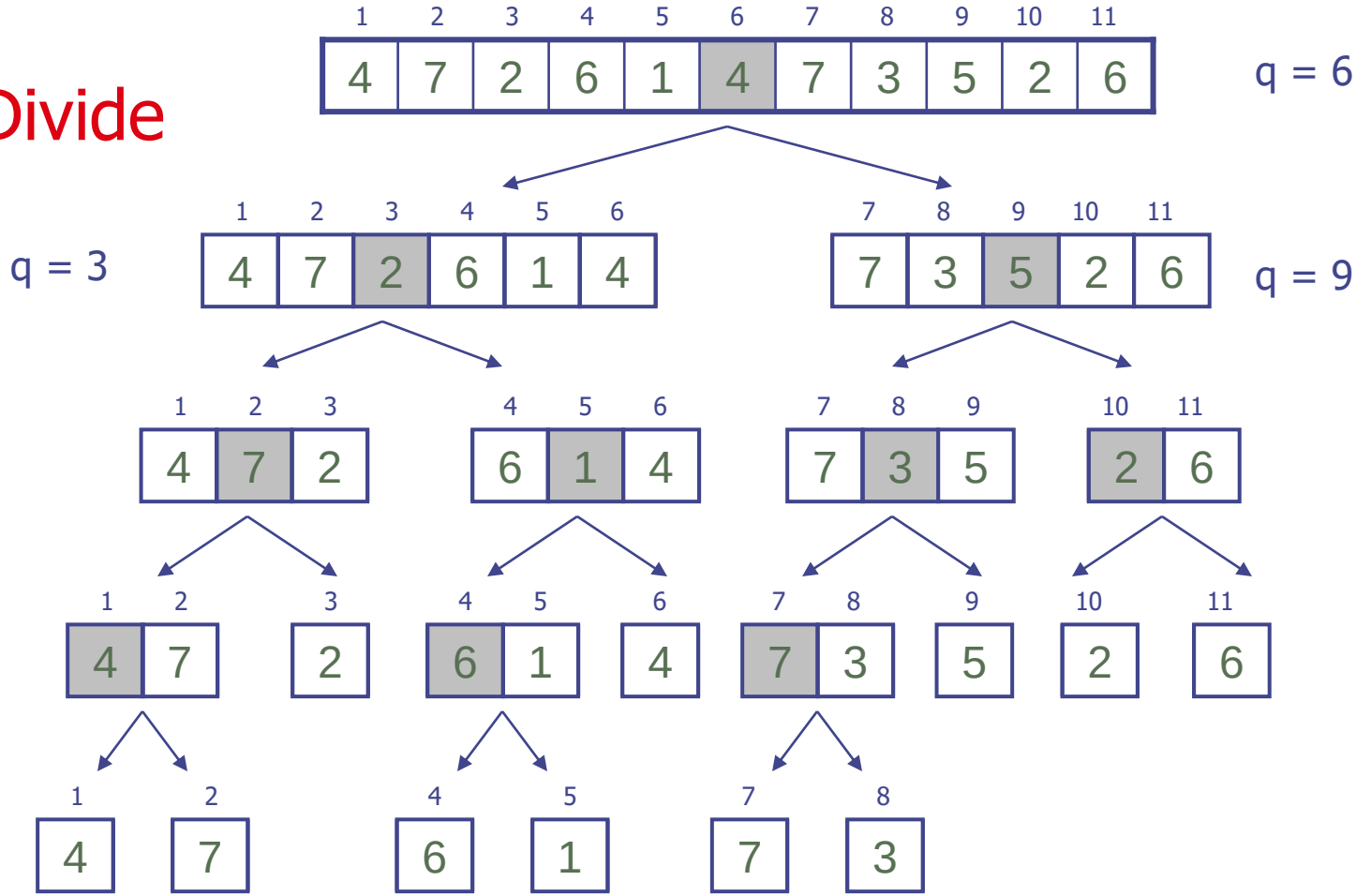
Example – n Power of 2

Conquer and Merge



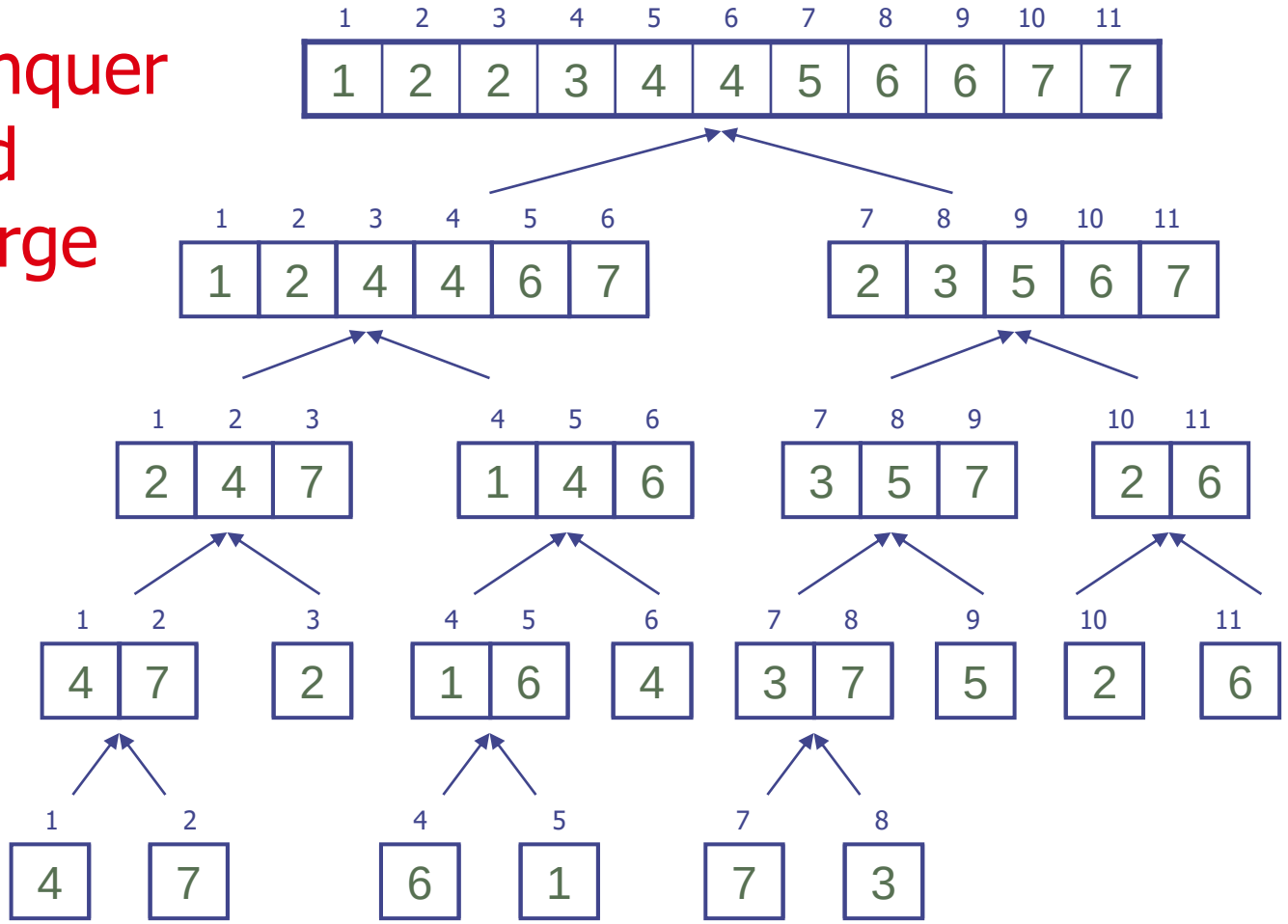
Example – n Not a Power of 2

Divide

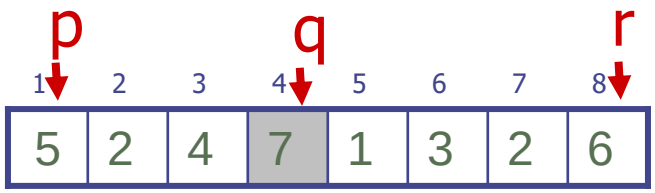


Example – n Not a Power of 2

Conquer and Merge



Merge Sort



MERGE-SORT(A, p, r)

if p < r ▷ Check for base case

  then $q \leftarrow \lfloor (p + r)/2 \rfloor$ ▷ Divide

    MERGE-SORT(A, p, q) ▷ Conquer

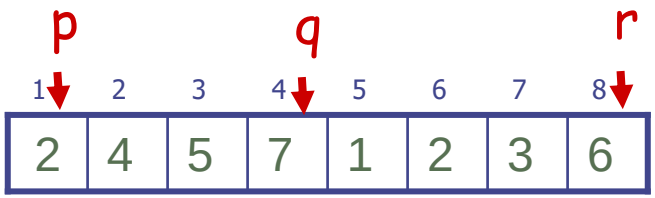
    MERGE-SORT(A, q + 1, r) ▷ Conquer

    MERGE(A, p, q, r) ▷ Combine

⑩ Initial call: MERGE-SORT(A, 1, n)

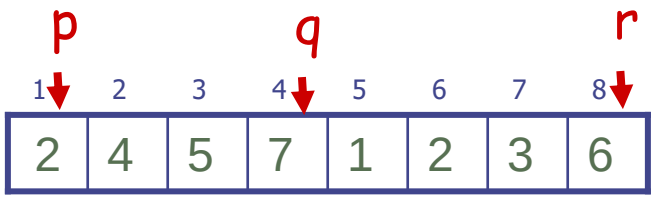
T(n) = 2 T(n/2) + ??

Merging



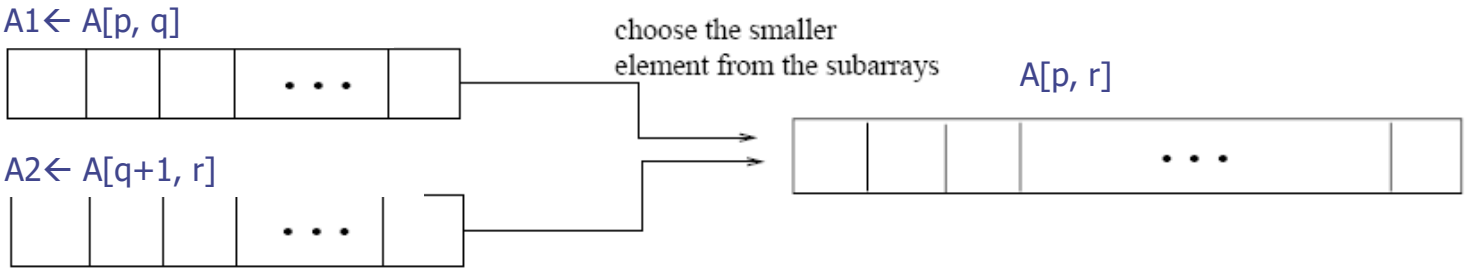
- **Input:** Array A and indices p, q, r such that $p \le q < r$
  - Subarrays $A[p\,..\,q]$ and $A[q+1\,..\,r]$ are sorted
- **Output:** One single sorted subarray $A[p\,..\,r]$

Merging



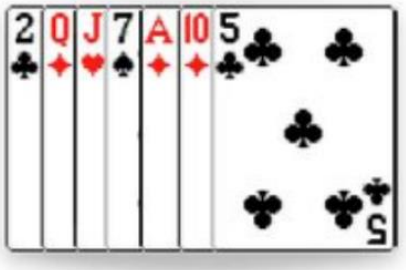
Idea for merging:

- Two piles of sorted cards
  - Choose the smaller of the two top cards
  - Remove it and place it in the output pile
- Repeat the process until one pile is empty
- Take the remaining input pile and place it

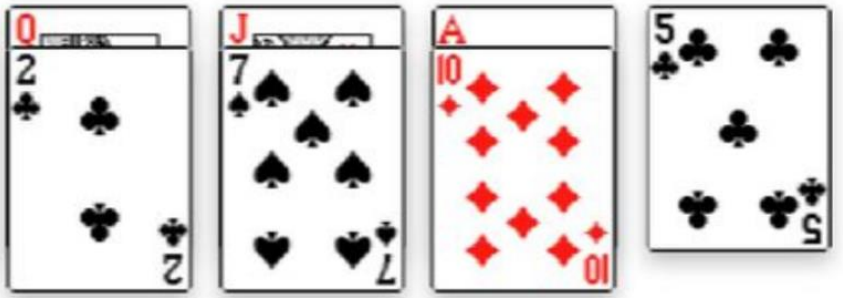


Example

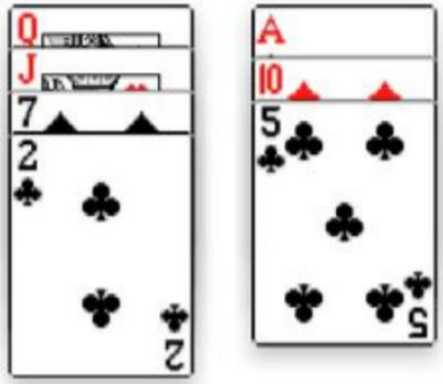
initial hand



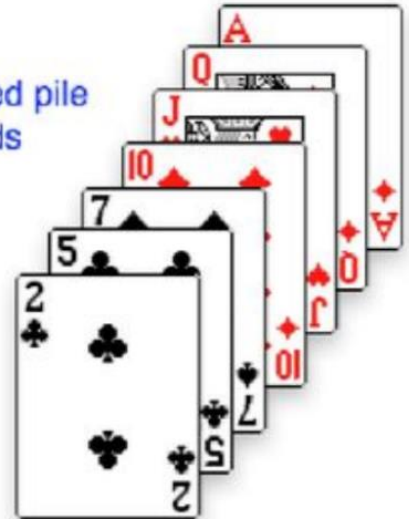
sorted piles of size two



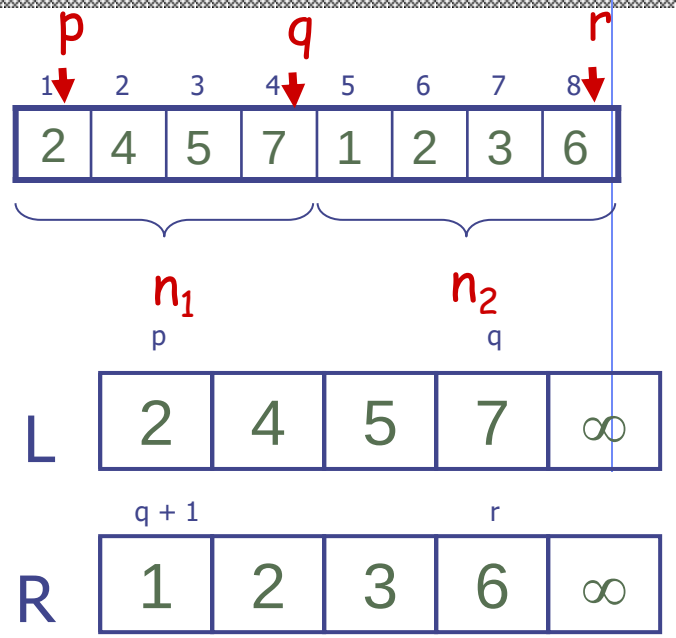
sorted piles of size four



final sorted pile of all cards

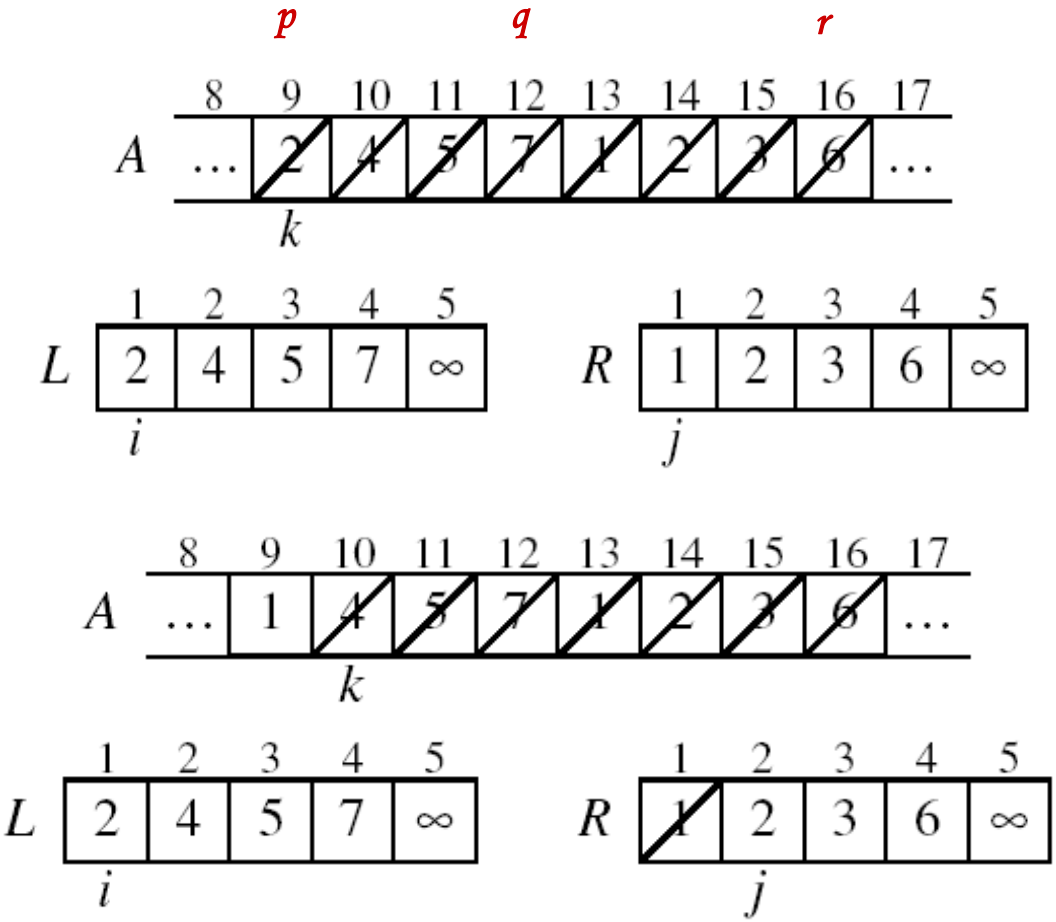


Merge - Pseudocode

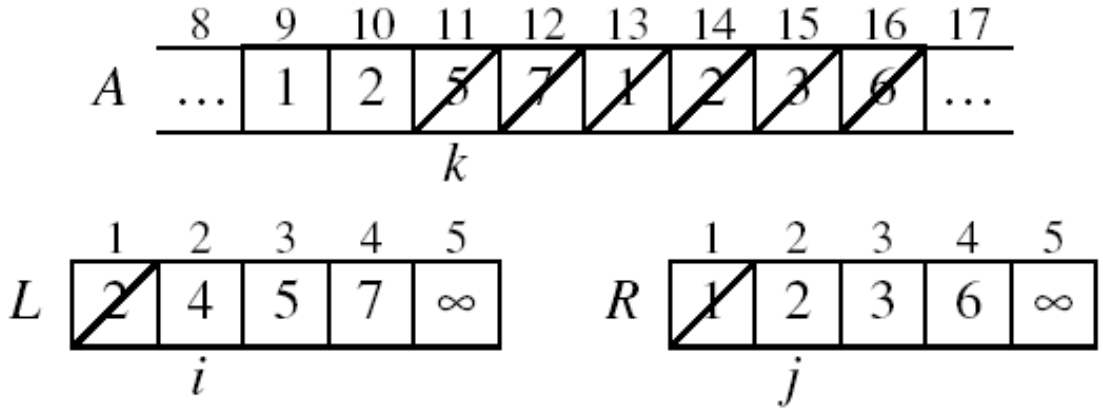


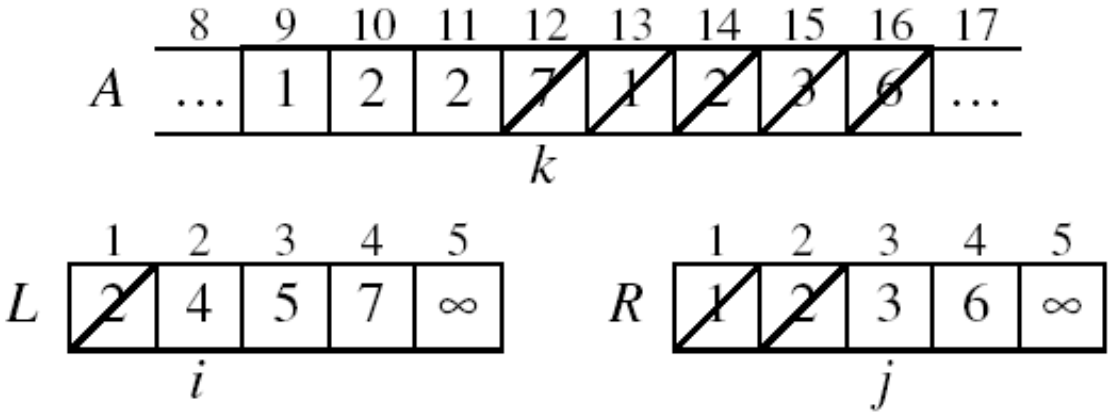

```
MERGE(A, p, q, r)
1. Compute n1 and n2
2. Copy the first n1 elements into
     L[1 . . n1 + 1] and  the next n2 elements into R[1
. . n2 + 1]
3. L[n1 + 1] ← ∞;        R[n2 + 1] ← ∞
4.  i ← 1;     j ← 1
5.  for k ← p to r
6.        do if L[ i ] ≤ R[ j ]
7.              then A[k] ← L[ i ]
8.                   i ←i + 1
9.              else A[k] ← R[ j ]
10.                  j ← j + 1
```

Example: MERGE(A, 9, 12, 16)

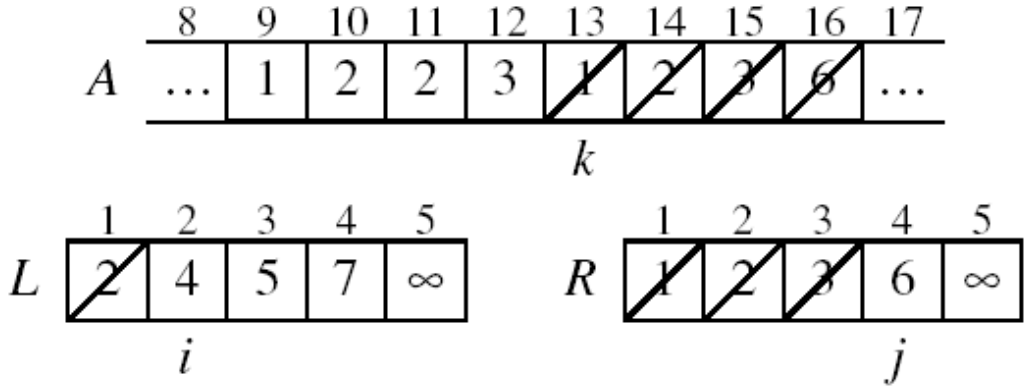


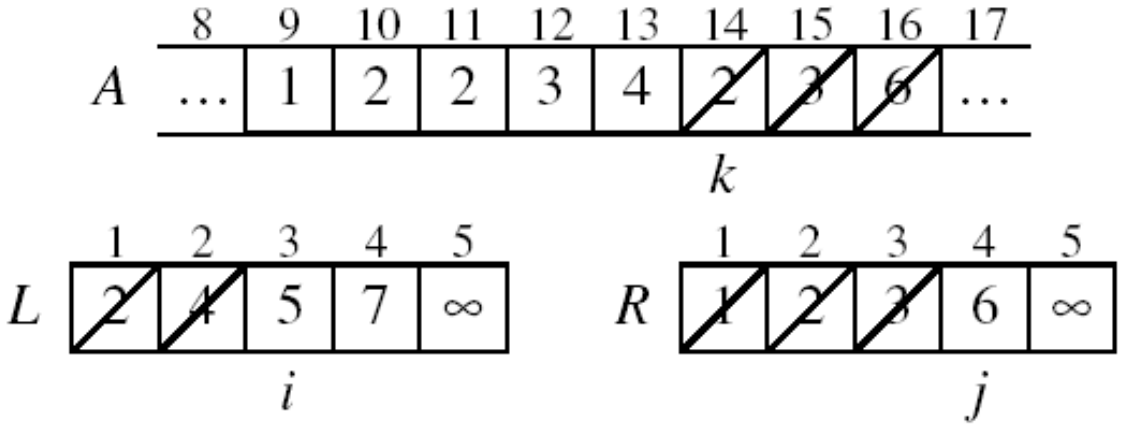
Example: MERGE(A, 9, 12, 16)



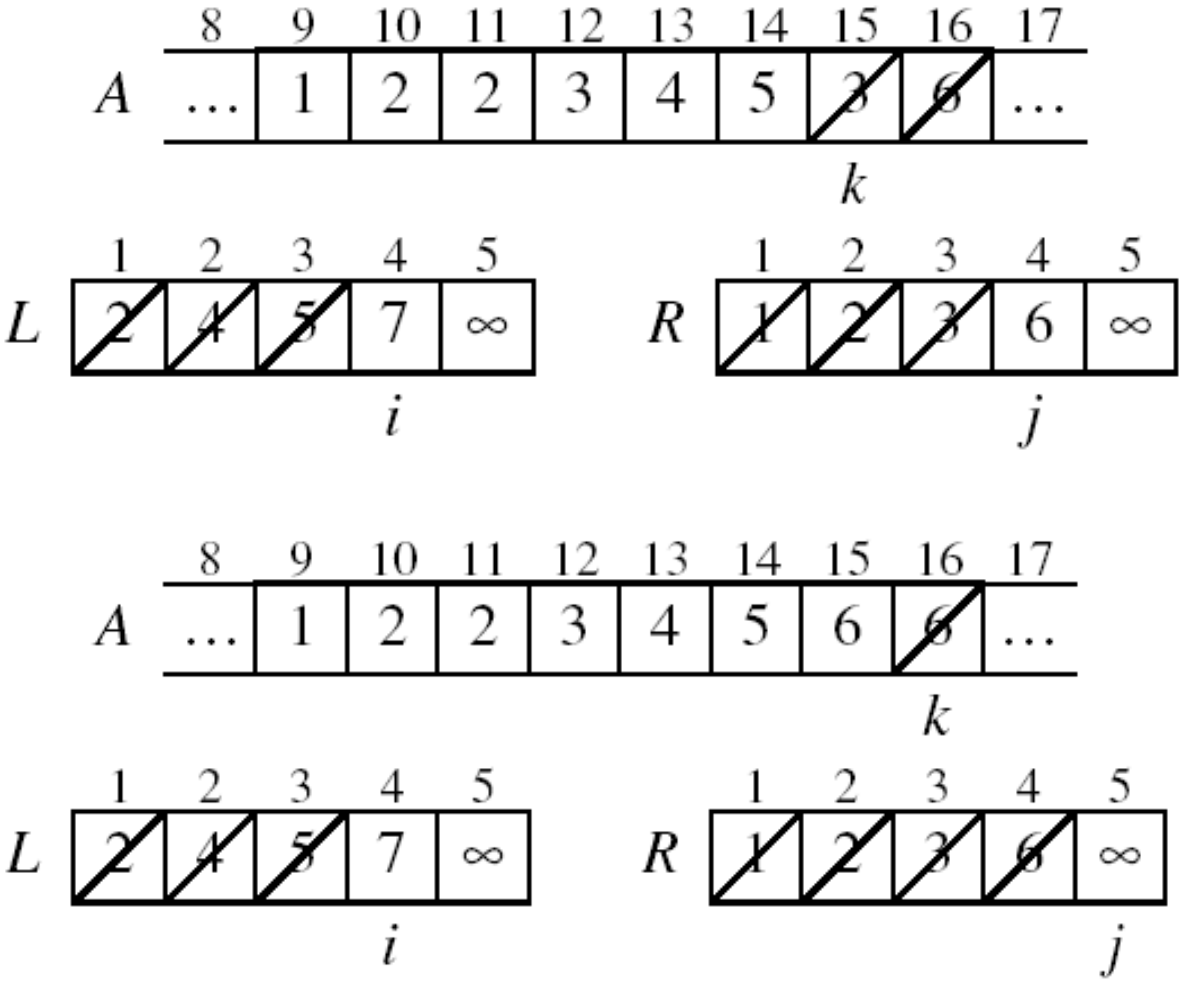


Example (cont.)

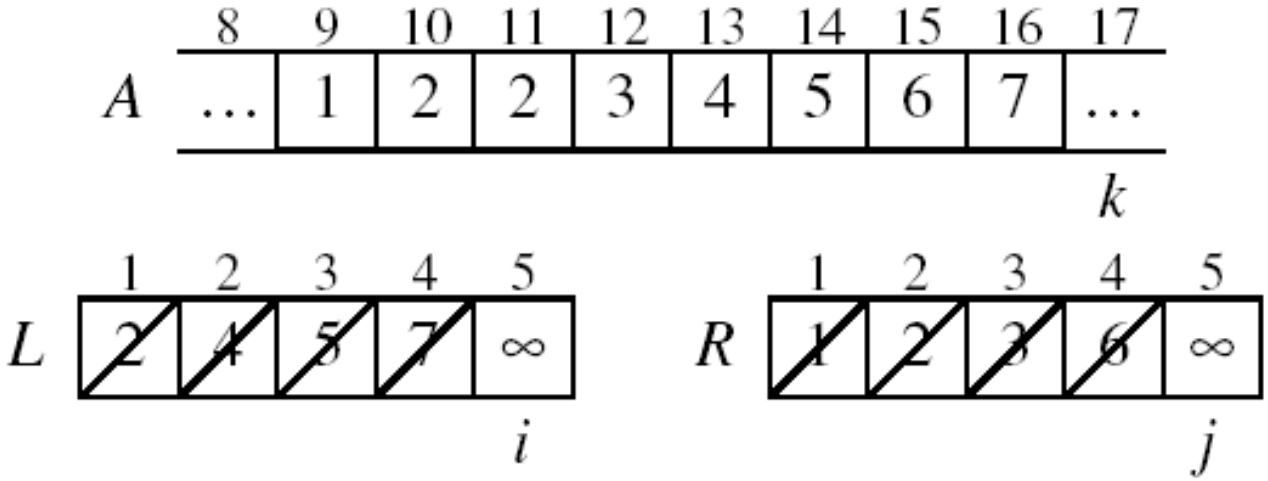




Example (cont.)



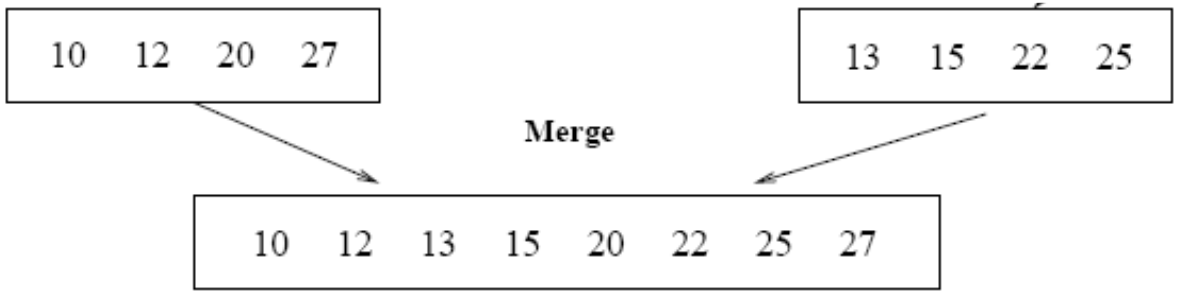
Example (cont.)



Done!

Running Time of Merge

- Initialization (copying into temporary arrays):
  - $\Theta(n_1 + n_2) = \Theta(n)$
- Adding the elements to the final array:
  - n iterations, each taking constant time $\Rightarrow \Theta(n)$
- Total time for Merge:
  - $\Theta(n)$



MERGE-SORT Running Time

- **Divide:**
  - compute q as the average of p and r: $D(n) = \Theta(1)$
- **Conquer:**
  - recursively solve 2 sub-problems, each of size $n/2 \Rightarrow 2T(n/2)$
- **Combine:**
  - MERGE on an n-element subarray takes $\Theta(n)$ time $\Rightarrow C(n) = \Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

$T(n)$ is $\Theta(n \log n)$